

CampusFind — Project Report

1. Cover Page

Project: CampusFind — Smart Lost & Found Matching System

Language: Python

Project Type: VITyarthi course project

Student: Naleen Brahmin

Registration Number: 26BCY10084

Course: Python Essentials

Faculty: Mohd Rafi Lone

Date: 26/09/2026

2. Introduction

CampusFind is a simple Python application designed to organize lost-and-found reports on a college campus. The application stores item information and uses a rule-based matching algorithm to identify potentially related lost and found reports.

3. Problem Statement

Manual lost-and-found communication can make it difficult to connect an owner with a found item. CampusFind provides a structured way to record reports and compare them.

4. Objectives

- Create a centralized record of lost and found items.
- Provide simple search and filtering.
- Implement a transparent matching algorithm.
- Provide an understandable match score.
- Store data persistently.
- Demonstrate modular Python programming and testing.

5. Functional Requirements

FR1 — Lost Item Registration

The system shall accept and store lost-item details.

FR2 — Found Item Registration

The system shall accept and store found-item details.

FR3 — Search

The system shall search records by keyword and item type.

FR4 — Smart Matching

The system shall compare a lost item with found items and calculate a score.

FR5 — Statistics

The system shall display counts of records and possible matches.

6. Non-Functional Requirements

Performance

The application should respond quickly for a small college dataset.

Usability

Menu options and prompts should be understandable to a beginner.

Reliability

Stored records should survive application restarts.

Maintainability

Features are separated into modules so individual components can be modified independently.

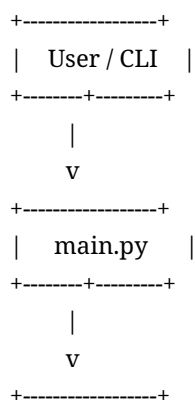
Error Handling

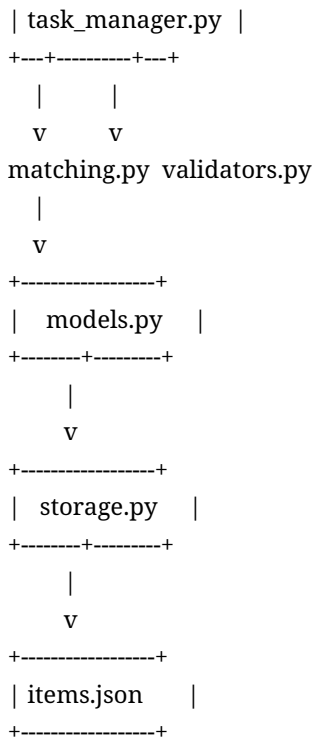
Invalid choices and missing values should not terminate the application unexpectedly.

Resource Efficiency

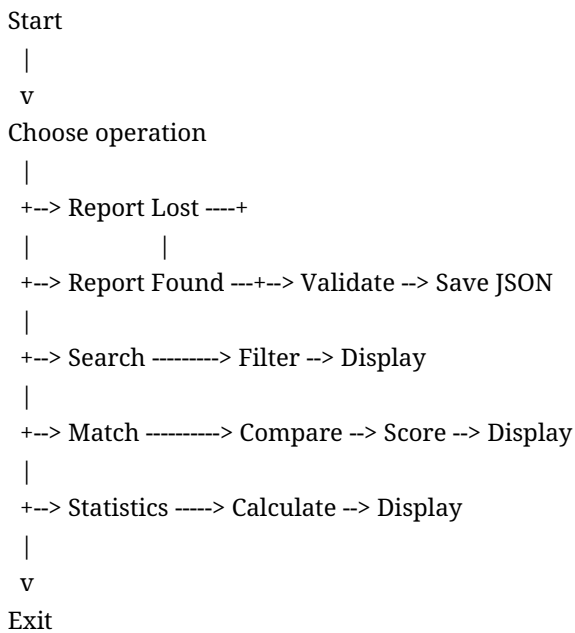
The project uses local JSON storage and standard Python libraries only.

7. System Architecture

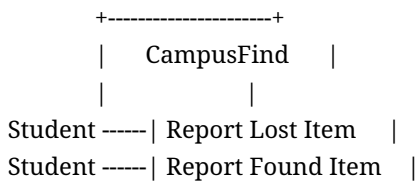




8. Workflow Diagram



9. Use Case Diagram



```

Student -----| Search Items      |
Student -----| Find Possible Match |
Student -----| View Statistics    |
               +-----+

```

10. Class / Component Design

Item

```

|— item_id
|— item_type
|— name
|— category
|— color
|— description
|— location
|— date
|— contact
|— status

```

LostFoundManager

```

|— create_item()
|— search()
|— find_matches()
|— statistics()

```

JsonStorage

```

|— load()
|— save()

```

Matching

```

|— calculate_match()

```

11. Sequence Diagram

```

User -> main.py: Enter lost item
main.py -> validators.py: Validate input
validators.py -> main.py: Valid data
main.py -> task_manager.py: create_item()
task_manager.py -> models.py: Create Item
task_manager.py -> storage.py: save()
storage.py -> items.json: Write record
items.json -> storage.py: Success
storage.py -> task_manager.py: Success
task_manager.py -> main.py: Item
main.py -> User: Display item ID

```

12. Matching Algorithm

The system assigns weights to five criteria:

- Category: 30 points
- Color: 15 points
- Location similarity: 20 points
- Description/name similarity: 25 points
- Same date: 10 points

The maximum score is 100. The system displays possible matches at or above the configured threshold.

13. Design Decisions & Rationale

Python

Python was selected because the project is small, readable, and easy to test.

JSON

JSON provides simple persistent storage without requiring database installation.

Rule-Based Matching

A weighted rule-based approach is transparent and easy to explain in an academic viva. Each score can be traced back to a specific criterion.

Modular Design

Separate files reduce complexity and make maintenance easier.

14. Implementation Details

The application is divided into:

- UI/menu handling
- data model
- persistence
- validation
- matching algorithm
- business logic
- utility functions
- automated tests

15. Testing Approach

The project uses Python's `unittest` framework.

Tests cover:

1. High similarity producing a strong match score.
2. Keyword search returning relevant records.
3. Invalid IDs producing controlled errors.

Command:

```
python -m unittest -v
```

16. Results

Expected behavior:

- Users can create lost and found reports.
- Records are saved to `data/items.json`.
- Searches return matching records.
- The matching engine displays possible matches with scores and explanations.
- Statistics summarize the stored dataset.

17. Challenges Faced

- Designing a useful matching score without machine learning.
- Handling incomplete or invalid user input.
- Keeping data persistent between program executions.
- Dividing the application into maintainable modules.

18. Learnings & Key Takeaways

- Modular Python programming
- Dataclasses and object-oriented design
- JSON persistence
- Basic text similarity
- Weighted scoring
- Unit testing
- Input validation
- Software documentation

19. Future Enhancements

- SQLite database
- Flask web application
- Login/authentication
- Image comparison
- Email/SMS notifications
- Administrative dashboard
- Cloud deployment

20. References

- [Python Standard Library documentation](#)
- [Python ``unittest`` documentation](#)
- [Python ``dataclasses`` documentation](#)